

Chapter 1 Exercise

XM 1.1 What are the three main purposes of an operating system?

The three main purposes of an operating system are:

1. **Convenience**
 - Makes the computer system easier to use.
 - Provides user interface, file systems, and program execution services.
 2. **Efficiency**
 - Manages hardware resources (CPU, memory, I/O devices) efficiently.
 - Ensures maximum utilization of system resources.
 3. **Ability to Evolve**
 - Allows the system to be extended with new features without disrupting existing services.
 - Supports new hardware and software technologies.
-

xm 1.2 We have stressed the need for an operating system to make efficient use of the computing hardware. When is it appropriate for the operating system to forsake this principle and to “waste” resources? Why is such a system not really wasteful?

It is appropriate to “waste” resources when:

- Improving **user experience**
- Increasing **system reliability**
- Enhancing **security**
- Supporting **real-time guarantees**

For example:

- Keeping extra memory unused for safety.
- Using additional CPU cycles to improve responsiveness.

Such a system is not truly wasteful because:

- The “wasted” resources improve performance quality, reliability, or usability.
- The trade-off results in better overall system value.

So, sometimes controlled inefficiency improves overall system effectiveness.

1.3 What is the main difficulty that a programmer must overcome in writing an operating system for a real-time environment?

The main difficulty is:

👉 **Guaranteeing strict timing constraints.**

Main difficulty (প্রধান সমস্যা)

👉 **Main difficulty হলো: Timing constraints নিশ্চিত করা (guaranteeing response within strict deadlines)**

অর্থাৎ programmer-কে নিশ্চিত করতে হবে:

- প্রতিটি task নির্দিষ্ট deadline-এর মধ্যে complete হবে
 - কোনো delay বা unpredictable execution হবে না
-

কেন এটা difficult

১. Multiple tasks manage করা

- অনেক process একসাথে চলতে পারে
- সবগুলোর deadline maintain করা কঠিন

২. Predictable scheduling দরকার

- OS-কে ঠিকভাবে decide করতে হবে কোন process কখন চলবে

৩. Hardware এবং interrupts handle করা

- Interrupts unpredictable হতে পারে

- কিন্তু real-time system-এ exact timing maintain করতে হয়

8. Resource sharing problem

- Multiple processes একই resource ব্যবহার করতে চাইলে delay হতে পারে

সহজ উদাহরণ

ধরো airbag system:

- Accident detect হলে airbag 0.02 second-এর মধ্যে open হতে হবে
- Delay হলে driver injured হতে পারে

👉 তাই strict timing guarantee করা খুব important।

1.4 Keeping in mind the various definitions of operating system, consider whether the operating system should include applications such as web browsers and mail programs. Argue both that it should and that it should not.

Argument: OS-এর মধ্যে application থাকা উচিত

১. User convenience বৃদ্ধি পায়

- যদি OS-এর মধ্যে browser এবং mail program built-in থাকে, user install না করেও সরাসরি ব্যবহার করতে পারে।
- এতে system ব্যবহার করা সহজ হয়।

২. Better integration

- OS-এর সাথে application tightly integrated থাকলে performance এবং compatibility ভালো হয়।
- উদাহরণ: file system, network, security সহজে ব্যবহার করা যায়।

৩. Beginners-এর জন্য helpful

- নতুন user-দের আলাদা করে software install করতে হয় না।
- System ready-to-use থাকে।

8. **Standard functionality নিশ্চিত করা যায়**

- সব system-এ basic applications available থাকে।
-

Argument: OS-এর মধ্যে application থাকা উচিত নয়

১. **OS complex হয়ে যায়**

- OS-এর main কাজ হলো hardware manage করা, application provide করা নয়।
- Extra applications যোগ করলে OS বড় এবং complex হয়ে যায়।

২. **Memory এবং storage বেশি ব্যবহার হয়**

- Built-in applications system resources বেশি ব্যবহার করে।

৩. **Flexibility কমে যায়**

- User নিজের পছন্দের browser বা mail program ব্যবহার করতে নাও পারতে পারে।
- User choice সীমিত হয়।

৪. **Maintenance এবং update difficult হয়**

- Application update করতে OS update করতে হতে পারে।

Conclusion:

An operating system should focus on **core resource management**, while applications should remain separate for flexibility and maintainability.

1.5 How does the distinction between kernel mode and user mode function as a rudimentary form of protection (security)?

প্রথমে বুঝি Kernel Mode এবং User Mode কী

◆ Kernel Mode

- এখানে OS-এর core অংশ চলে।
- সব hardware এবং system resources (CPU, memory, disk) সরাসরি access করা যায়।
- সব privileged instruction execute করা যায়।

◆ User Mode

- এখানে সাধারণ user program (browser, editor, game) চলে।
- সরাসরি hardware access করা যায় না।
- privileged instruction execute করা যায় না।

কীভাবে এটা Protection দেয়

১. Direct hardware access বন্ধ থাকে

User mode-এ থাকা program সরাসরি:

- memory modify করতে পারে না
- disk write করতে পারে না
- I/O device control করতে পারে না

👉 এতে malicious program system নষ্ট করতে পারে না।

২. Privileged instructions restrict করা হয়

কিছু instruction শুধু kernel mode-এ execute করা যায়।

যদি user program এগুলো execute করতে চায়:

- CPU trap generate করে
- control OS-এর হাতে চলে যায়

👉 এতে unauthorized access বন্ধ হয়।

৩. Memory protection

User program শুধু তার allocated memory access করতে পারে।

- অন্য program-এর memory access করলে error হবে
- kernel memory access করলে system block করবে

👉 এতে data protection নিশ্চিত হয়।

৪. System call মাধ্যমে controlled access

User program যখন hardware দরকার করে, তখন:

- সরাসরি access না করে
- system call ব্যবহার করে

Kernel verify করে request valid কি না, তারপর কাজ করে।

👉 এতে controlled এবং secure access হয়।

1.6 Which of the following instructions should be privileged?

Instruction	Privileged?	Reason
a. Set value of timer	☑ Yes	Could affect CPU scheduling or time slicing
b. Read the clock	✗ No	Safe for user programs to read
c. Clear memory	☑ Yes	Could erase critical OS or other process data
d. Issue a trap instruction	✗ No	Traps are designed to request OS service safely
e. Turn off interrupts	☑ Yes	Could disable OS control of system events
f. Modify entries in device-status table	☑ Yes	Critical hardware management
g. Switch from user to kernel mode	☑ Yes	Should be controlled by OS for security

Instruction	Privileged?	Reason
h. Access I/O device	☒ Yes	Direct device access must be controlled

xm 1.7 Some early computers protected the operating system by placing it in a memory partition that could not be modified by either the user job or the operating system itself. Describe two difficulties that you think could arise with such a scheme.

Early memory-partition protection – potential difficulties

1. **OS cannot modify itself:**
 - Updating or patching the OS in that partition becomes difficult or impossible.
2. **Limited flexibility:**
 - Cannot dynamically allocate memory for OS expansion or special processes.
 - Any changes require hardware-level intervention or reboot.

1.8 Some CPUs provide for more than two modes of operation .What are two possible uses of these multiple modes?

১. Better protection এবং security provide করার জন্য

Multiple modes ব্যবহার করে system-কে বিভিন্ন privilege level-এ ভাগ করা যায়।

উদাহরণ:

- Mode 0 → Kernel mode (সবচেয়ে বেশি privilege)
- Mode 1 → Device drivers mode
- Mode 2 → System services mode
- Mode 3 → User mode (সবচেয়ে কম privilege)

সুবিধা:

- সব program-কে full access দেওয়া হয় না

- যদি কোনো driver crash করে, পুরো kernel crash না-ও করতে পারে
- System বেশি secure হয়

👉 অর্থাৎ, sensitive কাজ শুধু higher privilege mode-এ করা যায়।

২. Virtualization support করার জন্য (Hypervisor mode)

Multiple modes ব্যবহার করে **virtual machine support** করা যায়।

এখানে:

- Hypervisor mode → Virtual machine control করে
- Kernel mode → Guest operating system run করে
- User mode → Guest applications run করে

সুবিধা:

- এক hardware-এ multiple operating system run করা যায়
 - Virtual machines safe এবং isolated থাকে
-

1.9 Timers could be used to compute the current time. Provide a short description of how this could be accomplished.

Timers ব্যবহার করে current time compute করা যায় কারণ timer hardware নির্দিষ্ট interval-এ interrupt generate করে, যা OS count করতে পারে।

নিচে সহজভাবে ব্যাখ্যা করা হলো:

কীভাবে timers ব্যবহার করে current time calculate করা যায়

1. Hardware timer regular interval-এ interrupt দেয়

- Timer configure করা হয় যেন প্রতি 1 millisecond বা 10 millisecond-এ interrupt দেয়।

2. Operating system interrupt count করে

- প্রতিবার interrupt হলে OS একটি counter increase করে।

3. Elapsed time calculate করা হয়

- যদি timer প্রতি 1 millisecond-এ interrupt দেয়, এবং counter = 5000 হয়
- তাহলে elapsed time = 5000 milliseconds = 5 seconds

4. Current time determine করা হয়

- System start time-এর সাথে elapsed time যোগ করে current time calculate করা হয়।

1.10 Give two reasons why caches are useful. What problems do they solve? What problems do they cause? If a cache can be made as large as the device for which it is caching (for instance, a cache as large as a disk), why not make it that large and eliminate the device?

১. Caches কেন useful (দুটি কারণ)

কারণ ১: Faster access speed

Cache হলো high-speed memory, যা frequently used data temporarily store করে।

- CPU cache থেকে data অনেক দ্রুত access করতে পারে
- RAM বা disk থেকে data আনার তুলনায় cache অনেক faster

👉 ফলে system performance improve হয়।

কারণ ২: CPU waiting time কমায়

যদি CPU প্রতিবার slow memory (RAM বা disk) থেকে data আনতে হয়, তাহলে CPU idle থাকতে পারে।

Cache frequently used data store করে রাখে, তাই CPU দ্রুত data পায়।

👉 এতে CPU efficiency বাড়ে।

২. Cache কী problem solve করে

Problem solved: Speed mismatch problem

CPU খুব fast, কিন্তু RAM এবং disk তুলনামূলক slow।

এই speed difference কে বলা হয় **memory latency problem**।

Cache এই problem solve করে কারণ:

- Frequently used data fast cache-এ রাখা হয়
- CPU slow memory access কম করতে হয়

👉 এতে overall system speed বাড়ে।

৩. Cache কী problem cause করে

Problem ১: Cache coherence problem

একই data cache এবং main memory-তে থাকলে inconsistency হতে পারে।

উদাহরণ:

- Cache-এর data update হয়েছে
- কিন্তু RAM-এর data update হয়নি

👉 এতে incorrect data পাওয়া যেতে পারে।

Problem ২: Extra cost

Cache memory fast technology দিয়ে তৈরি হয়, তাই expensive।

👉 Large cache বানানো costly।

Problem ৩: Complexity increase করে

Cache manage করার জন্য extra hardware এবং algorithm দরকার।

👉 System design complex হয়।

৪. Cache যদি device-এর মতো বড় বানানো যায়, তাহলে device remove করা হয় না কেন

যেমন: disk-এর মতো বড় cache কেন বানানো হয় না?

কারণ ১: Cache খুব expensive

Cache fast memory দিয়ে তৈরি, তাই disk-এর মতো বড় cache বানানো অত্যন্ত costly।

Disk অনেক cheaper।

কারণ ২: Cache volatile memory

Cache সাধারণত volatile memory।

Power off হলে data lost হয়ে যায়।

Disk non-volatile memory, power off হলেও data থাকে।

কারণ ৩: Disk long-term storage provide করে

Cache temporary storage।

Disk permanent storage।

xm 1.11 Client-server vs Peer-to-peer models in distributed systems

Aspect	Client-Server	Peer-to-Peer (P2P)
Structure	Centralized server provides resources	All nodes (peers) share resources equally
Resource management	Server controls access and consistency	Each peer manages its own resources
Communication	Clients request, server responds	Nodes communicate directly, can act as client and server
Example	Web servers, email servers	Torrent networks, blockchain

Summary: Client-server is centralized and easier to control, while peer-to-peer is decentralized and scalable.

Here are your answers with the full questions included, written clearly and completely:

1.12 How do clustered systems differ from multiprocessor systems? What is required for two machines belonging to a cluster to cooperate to provide a highly available service?

১. Clustered system এবং Multiprocessor system-এর পার্থক্য

বিষয়	Multiprocessor System	Clustered System
গঠন	একটিমাত্র কম্পিউটারের ভেতরে একাধিক CPU	একাধিক স্বাধীন কম্পিউটার (node)
Memory	Shared memory (সব CPU একই memory share করে)	প্রতিটি node-এর আলাদা memory
Communication	Internal bus বা shared memory	Network-এর মাধ্যমে যোগাযোগ
Control	একটিমাত্র operating system	প্রতিটি node-এ আলাদা OS থাকতে পারে

বিষয়	Multiprocessor System	Clustered System
Fault tolerance	তুলনামূলক কম	High availability design করা যায়

সহজভাবে:

- **Multiprocessor system** = এক মেশিন, অনেক CPU
- **Clustered system** = অনেক মেশিন, network দিয়ে যুক্ত

২. High availability দেওয়ার জন্য cluster-এ কী দরকার

Cluster-এর মূল লক্ষ্য হলো:

👉 যদি একটি machine fail করে, অন্যটি সাথে সাথে service continue করবে।

এটা করার জন্য নিচের জিনিসগুলো দরকার:

১. Reliable communication

- Node গুলোর মধ্যে stable এবং fast network থাকতে হবে
- একে অপরের status জানার জন্য heartbeat signal দরকার

২. Data sharing বা replication

- সব node-এ একই data থাকতে হবে
- Shared storage বা data replication ব্যবহার করা হয়
- না হলে failover-এর সময় data mismatch হবে

৩. Failover mechanism

- যদি একটি node crash করে
- অন্য node automatically service takeover করবে

👉 এটাকে failover বলে।

8. Cluster management software

- Node monitoring
- Load balancing
- Resource management
- Failure detection

সব coordinate করার জন্য special cluster software দরকার।

1.13 Consider a computing cluster consisting of two nodes running a database. Describe two ways in which the cluster software can manage access to the data on the disk. Discuss the benefits and disadvantages of each.

1. Shared Disk Approach

- Both nodes access the **same disk storage**.
- **Benefits:**
 - Data is always consistent.
 - Simplifies backup and central management.
- **Disadvantages:**
 - Requires strict coordination (locking) to avoid data conflicts.
 - Single disk failure can impact both nodes.

2. Replicated Database Approach

- Each node maintains its **own copy of the data**.
 - **Benefits:**
 - High availability – if one node fails, the other continues.
 - Reduces contention for disk access.
 - **Disadvantages:**
 - Must keep all copies synchronized (complex).
 - Higher storage requirements.
-

1.14 What is the purpose of interrupts? How does an interrupt differ from a trap? Can traps be generated intentionally by a user program? If so, for what purpose?

Purpose of interrupts:

- Allow **hardware devices** to notify the CPU when they need attention.
- Improve system efficiency by avoiding the need for constant polling of devices.

Difference between interrupt and trap:

Feature	Interrupt	Trap
Source	External hardware event	Internal software or exception
Purpose	Notify CPU of hardware events (e.g., I/O complete)	Handle exceptional conditions or system calls
Can be generated by user	No	Yes

Traps generated intentionally by a user:

- To invoke **system calls** safely (e.g., `read()`, `write()`).
- To handle exceptions like invalid memory access or division by zero.

1.15 Explain how the Linux kernel variables HZ and jiffies can be used to determine the number of seconds the system has been running since it was booted.

- **HZ**: Number of timer ticks per second (frequency of timer interrupts).
- **jiffies**: Total number of timer ticks since system boot.

Calculation:

$$\text{System uptime in seconds} = \frac{\text{jiffies}}{\text{HZ}}$$

- Example: If $\text{HZ} = 100$ and $\text{jiffies} = 500000$, then $\text{uptime} = 500000 \div 100 = 5000$ seconds.
 - This allows Linux to track system uptime without requiring a separate real-time clock.
-
-

1.16 Direct memory access (DMA) is used for high-speed I/O devices in order to avoid increasing the CPU's execution load.

a. How does the CPU interface with the device to coordinate the transfer?

Step-by-step process:

1. CPU DMA controller-কে initialize করে

CPU নিচের তথ্যগুলো DMA controller-কে দেয়:

- Source address (data কোথা থেকে আসবে, যেমন device)
- Destination address (data কোথায় যাবে, যেমন memory)
- কত byte transfer হবে
- Transfer direction (read বা write)

2. DMA controller transfer শুরু করে

- DMA controller নিজে নিজে memory এবং device-এর মধ্যে data transfer করে
- CPU-এর involvement দরকার হয় না

👉 এতে CPU free হয়ে অন্য কাজ করতে পারে।

b. How does the CPU know when the memory operations are complete?

DMA controller transfer শেষ হলে CPU-কে signal দেয়।

কীভাবে signal দেয়:

- DMA controller একটি interrupt generate করে
- Interrupt পাওয়ার পর CPU বুঝতে পারে transfer complete হয়েছে
- তারপর CPU next operation শুরু করতে পারে

c. The CPU is allowed to execute other programs while the DMA controller is transferring data. Does this process interfere with the execution of the user programs? If so, describe what forms of interference are caused.

👉 হ্যাঁ, সামান্য interfere করতে পারে, কিন্তু overall benefit বেশি।

Interference-এর কারণ: Bus contention

- CPU এবং DMA controller দুজনেই memory access করার জন্য system bus ব্যবহার করে
- DMA যখন data transfer করে, তখন CPU কিছু সময় memory access করতে delay পেতে পারে

Result:

- CPU execution সামান্য slow হতে পারে
 - কিন্তু overall system performance improve হয়, কারণ CPU data transfer করতে ব্যস্ত থাকে না
-

1.17 Some computer systems do not provide a privileged mode of operation in hardware. Is it possible to construct a secure operating system for these computer systems? Give arguments both that it is and that it is not possible.

Argument that it is possible:

১. Software-based protection ব্যবহার করা যায়

- Operating system software level-এ check implement করতে পারে।
- OS verify করতে পারে program কী করতে পারবে আর কী পারবে না।

২. Careful program design

- Programs-কে এমনভাবে design করা যায় যাতে তারা restricted environment-এ run করে।
- Example: interpreter বা virtual machine ব্যবহার করা।

৩. Controlled execution environment

- OS সব access request monitor করতে পারে।
- Unauthorized access detect করে block করা যায়।

👉 এতে কিছু level পর্যন্ত security provide করা সম্ভব।

Argument that it is not possible:

১. Hardware protection না থাকলে full control সম্ভব নয়

- User program সরাসরি hardware access করতে পারে।
- OS prevent করতে পারবে না।

২. OS memory modify করা সম্ভব

- Malicious program OS-এর code overwrite করতে পারে।
- এতে পুরো system compromise হয়ে যেতে পারে।

৩. Privileged instructions restrict করা যায় না

- Hardware support না থাকলে sensitive instruction execute করা বন্ধ করা যায় না।

৪. Complete security guarantee করা যায় না

- Buggy বা malicious program পুরো system damage করতে পারে।

1.18 Many SMP systems have different levels of caches; one level is local to each processing core, and another level is shared among all processing cores. Why are caching systems designed this way?

এই প্রশ্নে বলা হচ্ছে: কেন **SMP (Symmetric Multiprocessing) system**-এ multiple level cache থাকে, যেখানে কিছু cache প্রতিটি core-এর জন্য আলাদা (local) এবং কিছু cache সব core-এর মধ্যে shared থাকে?

নিচে সহজভাবে ব্যাখ্যা করা হলো।

১. Local Cache (L1 / L2) কেন দরকার

◆ খুব দ্রুত access-এর জন্য

- প্রতিটি core-এর নিজস্ব ছোট cache থাকে।

- এতে core তার frequently used data খুব দ্রুত পায়।
- Main memory থেকে data আনতে দেরি হয় না।

◆ Latency কমানোর জন্য

- Local cache core-এর খুব কাছাকাছি থাকে।
- তাই access time খুব কম।

◆ Parallel efficiency বাড়ায়

- প্রতিটি core নিজের data independently handle করতে পারে।
- অন্য core-এর জন্য অপেক্ষা করতে হয় না।

২. Shared Cache (L3 বা তার উপরে) কেন দরকার

◆ Shared data efficiently manage করার জন্য

- যদি একাধিক core একই data ব্যবহার করে, shared cache থেকে দ্রুত পাওয়া যায়।
- বারবার main memory access করতে হয় না।

◆ Memory traffic কমানোর জন্য

- Main memory slow।
- Shared cache থাকলে memory access কম হয়।

◆ Cache coherence সহজ করার জন্য

- Multiple core একই data modify করলে consistency maintain করতে shared cache সাহায্য করে।

৩. কেন hierarchical (স্তরভিত্তিক) design করা হয়

এই design তিনটি জিনিস balance করে:

✓ Speed

- Local cache → খুব দ্রুত access

✓ Cost

- বড় fast cache খুব expensive
- তাই ছোট fast cache (L1), বড় কিন্তু একটু slow cache (L3)

✓ Scalability

- Multiple core efficiently কাজ করতে পারে
- System performance scale করা যায়

○

1.19 Rank the following storage systems from slowest to fastest:

1. **Magnetic tapes** – sequential access, very slow.
2. **Optical disk** – slower random access.
3. **Hard-disk drives** – faster than optical disks, but mechanical.
4. **Nonvolatile memory (NVM/SSD)** – faster than HDD, no moving parts.
5. **Main memory (RAM)** – very fast, volatile.
6. **Cache** – faster than main memory.
7. **Registers** – fastest, inside CPU.

Order (slowest → fastest):

Magnetic tapes < Optical disk < Hard-disk drives < Nonvolatile memory < Main memory < Cache < Registers

1.20 Consider an SMP system similar to the one shown in Figure 1.8. Illustrate with an example how data residing in memory could in fact have a different value in each of the local caches.

- **Scenario:** Two CPU cores (Core 1 and Core 2) both cache the same memory location x .
 1. Core 1 reads $x = 5$ into its local cache.
 2. Core 2 also reads $x = 5$ into its local cache.
 3. Core 1 updates $x = 10$ in its local cache but hasn't written back to main memory yet.
 4. Core 2 still has $x = 5$ in its cache.

- **Result:**
 1. Main memory might still have $x = 5$.
 2. Core 1 cache has $x = 10$.
 3. Core 2 cache has $x = 5$.
 - **Reason:** This is due to **cache coherence issues** in SMP systems, which require protocols (e.g., MESI) to ensure consistency.
-
-

1.21 Discuss, with examples, how the problem of maintaining coherence of cached data manifests itself in the following processing environments:

Cache coherence problem মানে হলো: একই data-এর multiple copy যদি cache-এ থাকে, এবং একটি copy update হয়, তাহলে অন্য copy-গুলো outdated (stale) হয়ে যেতে পারে। এখন তিন ধরনের system-এ এই problem কীভাবে হয়, উদাহরণসহ ব্যাখ্যা করা হলো।

a. Single-Processor Systems

Problem কীভাবে হয়

Single processor system-এ সাধারণত coherence problem কম হয়, কারণ শুধু একটিমাত্র CPU থাকে।

কিন্তু problem হতে পারে যখন I/O device সরাসরি memory update করে (DMA ব্যবহার করে)।

Example

- CPU cache-এ variable $X = 5$ আছে
- DMA ব্যবহার করে disk controller memory-এ $X = 10$ update করলো
- কিন্তু CPU cache-এ এখনো $X = 5$ আছে

👉 CPU outdated (stale) data পড়বে।

Solution

- Cache flush করা (cache clear করা)

- Write-through cache ব্যবহার করা
 - DMA শেষ হলে interrupt দিয়ে cache update করা
-

b. Multiprocessor Systems

Problem কীভাবে হয়

Multiprocessor system-এ multiple CPU/core থাকে, এবং প্রতিটি core-এর নিজস্ব cache থাকে।

একই memory location-এর multiple cache copy থাকতে পারে।

Example

ধরো:

- Core 1 cache-এ $X = 5$
- Core 2 cache-এ $X = 5$

এখন Core 1 update করলো:

- $X = 10$

কিন্তু Core 2 এখনো $X = 5$ ব্যবহার করছে।

👉 Data inconsistency problem হচ্ছে।

Solution

Cache coherence protocols ব্যবহার করা হয়, যেমন:

- MESI protocol
- MOESI protocol

এই protocol ensure করে সব cache updated থাকে।

c. Distributed Systems

Problem কীভাবে হয়

Distributed system-এ multiple computer (node) থাকে, এবং প্রতিটির নিজস্ব memory এবং cache থাকে।

Network-এর মাধ্যমে data share করা হয়, তাই coherence maintain করা আরও কঠিন।

Example

- Node A একটি shared file update করলো
- Node B এখনো old version cache থেকে পড়ছে

👉 Node B outdated data ব্যবহার করছে।

Solution

- Data replication synchronization
 - Write-invalidate protocol
 - Update propagation
 - Distributed cache coherence protocols
-

1.22 Describe a mechanism for enforcing memory protection in order to prevent a program from modifying the memory associated with other programs.

Memory protection enforce করার মূল লক্ষ্য হলো: একটি program যেন অন্য program-এর memory access বা modify করতে না পারে। এটা hardware এবং OS একসাথে implement করে।

নিচে প্রধান mechanism গুলো সহজভাবে ব্যাখ্যা করা হলো।

১. Base Register এবং Limit Register ব্যবহার করে

এটা সবচেয়ে basic এবং গুরুত্বপূর্ণ method।

কীভাবে কাজ করে

প্রতিটি process-এর জন্য hardware দুইটি register ব্যবহার করে:

- **Base register** → process-এর memory শুরু কোথা থেকে
- **Limit register** → process কতটুকু memory ব্যবহার করতে পারবে

CPU যখন কোনো memory access করে, hardware check করে:

Valid range:

$\text{Base address} \leq \text{Memory address} < \text{Base} + \text{Limit}$

যদি address এই range-এর বাইরে হয়:

- 👉 Hardware একটি trap (error interrupt) generate করে
- 👉 Operating system illegal access block করে

Example

ধরো:

- Base register = 1000
- Limit register = 500

Valid memory range = 1000 থেকে 1499

এখন যদি program access করতে চায়:

- Address = 1200 → Allowed ☒
- Address = 2000 → Not allowed ☒ (trap হবে)

২. Paging ব্যবহার করে memory protection

Paging system-এ memory ছোট ছোট page-এ ভাগ করা হয়।

প্রতিটি page-এর সাথে protection bit থাকে:

- Read-only
- Read-write
- Execute-only

Example:

- Code page → read-only
- Data page → read-write

👉 এতে program protected থাকে।

৩. Segmentation ব্যবহার করে protection

Memory বিভিন্ন segment-এ ভাগ করা হয়, যেমন:

- Code segment
- Data segment
- Stack segment

প্রতিটি segment-এর access permission থাকে।

Example:

- Code segment → read and execute only
 - Data segment → read and write
-

কেন এই protection দরকার

এতে prevent করা যায়:

- এক program অন্য program-এর data modify করা
- System crash হওয়া
- Security attack হওয়া

1.23 Which network configuration—LAN or WAN—would best suit the following environments?

a. A campus student union / A neighborhood → LAN

- Localized area, high-speed communication, limited distance.

b. Several campus locations across a statewide university system → WAN

- Wide geographic coverage, connects multiple LANs, slower than LAN but suitable for long distances.
-

1.24 Describe some of the challenges of designing operating systems for mobile devices compared with designing operating systems for traditional PCs.

- **Limited resources:** Less CPU, RAM, and battery.
 - **Power management:** OS must optimize for battery life.
 - **Wireless connectivity:** Frequent network changes and intermittent signals.
 - **Security:** Mobile devices are more exposed to theft or malware.
 - **User interface:** Touch-based, small screens, and varying resolutions.
 - **App sandboxing:** Each app must be isolated for security and stability.
-

1.25 What are some advantages of peer-to-peer systems over client–server systems?

- **No central server required,** reducing cost and single-point failure.
 - **Scalable:** Adding more peers increases both resources and users.
 - **Efficient resource sharing:** Each peer contributes storage, CPU, or bandwidth.
 - **Example:** Torrent file-sharing networks.
-

1.26 Describe some distributed applications that would be appropriate for a peer-to-peer system.

- **File sharing networks:** e.g., BitTorrent
 - **Content distribution networks (CDNs)** for multimedia
 - **Blockchain and cryptocurrencies:** e.g., Bitcoin, Ethereum
 - **Collaborative applications:** e.g., decentralized chat systems or collaborative editing tools
 - **Distributed computing projects:** e.g., SETI@home, Folding@home
-

xm 1.27 Identify several advantages and several disadvantages of open-source operating systems. Identify the types of people who would find each aspect to be an advantage or a disadvantage.

Advantages:

1. **Free of cost** – attractive for students, hobbyists, and organizations.
2. **Customizable** – developers and system administrators can modify OS to fit their needs.
3. **Transparency and security** – security experts can inspect code.
4. **Community support** – collaborative development and forums for help.

Disadvantages:

1. **Lack of official support** – businesses may need paid support for critical systems.
2. **Compatibility issues** – some proprietary software may not run on open-source OS.
3. **Steeper learning curve** – new users may struggle without formal documentation.

Who benefits:

- Students, developers, tech-savvy users → **advantages**
 - Businesses relying on commercial software or needing guaranteed support → **disadvantages**
-